

L - 1 Software Testing

18 August 2025 23:38

Software testing - Process of finding the bugs in order to give the software with good quality.

- Process of finding the defects in the software; Defects mean fault in the software which does not match the functional requirements.
- Verifying the functionality of application against the requirement specification.

Types of software testing:

1. **Manual testing** - testing the application manually without any automation tools.
 - Testing the software or functionality of software manually by using different testing methods.

Drawbacks:

- We do testing and regression test again and again.
- Same test cases need to be tested multiple times.

Automation - The technique or logic of converting a particular process to run on its own without any manual intervention.

2. **Automation testing** - It is the process of converting manual test cases into automation scripts with the help of automation tool and programming language.

When we use automation testing -

- Long term project
- application is complex
- more regression testing is required
- good quality of product is required

Disadvantages - Video, audio, captcha cannot be tested by automation testing

Advantages
• It is faster • It is more reliable and improves quality of application. • Parallel execution is easy
Disadvantages
• We cant test video, audio, captcha and network failure errors. • Exploratory testing is not possible. • 100% automation is not possible.

Difference between manual testing and automation testing

Manual Testing	Automation Testing
Test cases are executed by humans	Automation tool will execute test cases
Test execution is slow	Test execution is faster
Exploratory testing is possible	Exploratory testing is not possible
Not accurate as humans are prone to errors	More reliable
Trivial changes in UI will not be costly	Trivial changes in UI is costly
Test report visibility is not good	Test report visibility is good
Parallel execution needs more efforts	Parallel execution is easy
Execution during night is not possible	Execution during night is possible
Programming knowledge is not required	Programming knowledge is required
Boring	Not boring

Playwright

- It is developed by Microsoft
- It's supports all modern rendering engines including Chromium, WebKit and Firefox; tests on Windows, Linux and MacOS
- It gives test report too

Prerequisites - node.js, npm package, Visual Studio code

Procedure

- Create a folder
- Open VS code
- Open a file in the folder
- Terminal commands :
 - to run playwright - `npm init playwright@latest`
 - to run test cases - `npx playwright test` - it runs in the headless mode by default
 - `npx playwright test --headed`.

Headless - The working of the webpage is not visible

Headed - The working that is visible on the webpage

L - 2 Playwright Test Execution

21 August 2025 14:15

Playwright extension - <filename>.spec.ts / <filename>.test.ts

Code Explanation:

```
import {test} from 'playwright/test';
test("sample login test", async({page})=>{
  await page.goto('https://www.facebook.com/');
  await page.locator('#email').fill('vaidehi');
  await page.locator('id=pass').fill('qwerty');
  await page.locator('button[type = "submit"]').click();
  //tagname[attrname = 'attrvalue']
});
```

`{test}` -> Test object

`test()` -> Test function

Has 2 Parameters -

- "sample login test" -> Test script name
- `async()` -> Callback function

`page` -> It is the object given by the test function

`await` -> For making the functions to wait till the required parameters have been fetched

Methods

- `page.goto('<URL Link>')` - Open the given URL (Particular web application) in a particular web browser.
- `page.locator()` - Strategy used to find the **web elements**

Web application - Application that opens in a browser OR Combination of multiple web pages

Web elements - Anything that is displayed on the webpage

✳ There are 14 ways to find web elements in playwright; In selenium there are 8

Locating strategies

- For inspecting the elements - Ctrl + F

- By ID - `id-attrvalue`
- By CSS selector - `#idname`, `.classname`
- When the attribute value contains combination of character, special characters, and numbers, one should not use the attribute value to get the element as the id changes dynamically
Syntax - `tagname[type = 'attrvalue']`
- When there are multiple elements with same tagname then the method to fetch a specific element - **xpath** (4 ways)

Xpath - It is used to locate any element on any web page

1. xpath by visible text - `//tagname[text() = 'visible text']`

2. xpath by attribute - `//tagname[@attrname = 'attrvalue']`
 - Indexing (if multiple elements with same attribute - `(//tagname[@attrname = 'attrvalue'])[indexno]`
 - **Drawback** - If the element is removed the indexing will be changed the test script execution will be failed.
 - **Solution** - **Dynamic indexing**
3. xpath by contains - Usage reason - When the attribute value is long or changes dynamically - `//tagname[contains(@attrname, 'partialFixedattrvalue')]`
4. xpath by contains by visible text - Usage reason - When the text is long `//tagname[contains(text(), 'partialtext')]`

Dynamic Indexing - It is used for **dynamic web element**.

Dynamic web element - Element whose value always changes.

Procedure

1. Find the fixed element and locate it
2. Go back to the immediate common parent of both the elements -
Syntax - `/..` - for every previous element; as many previous element one goes as many times this should be written and locate it too
 - * Last common parent - `<html>` tag
 - **Common fixed element -> Required element**

- `locator('<elementlocation>').fill('<value>')` - To fill data in a given location
 - **Short hand** - `.fill('<elementlocation>', '<value>')`
- `locator('<elementlocation>').click()` - for the fetching the clickable element data.
 - **Short hand** - `.click('<elementlocation>')`

⇒ To run specific test case - `npx playwright test <filename>.spec.ts`

L - 3 Browser Launching, Web Context, Selecting Dropdowns & Buttons

22 August 2025 10:42

Browser Launching from the script

Concept - In general the playwright tests work on the web browser through the configurations given in the configuration file. Below code gives one an option to set the required web browser through the script.

Code Explanation:

```
import {test, chromium, webkit, firefox} from 'playwright/test';
test("Launching browser", async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.instagram.com/');
})
```

`async()` - It doesn't require page parameter as we going to create the page on our own.

`chromium, webkit, firefox` - For launching the browsers one needs to import the required browsers

- `<browsername>.launch({<parameters>})` - Launch should be attached to the browser in which one wants to open the web page. In the launch method in which mode browser needs to be run those parameters are given

`{headless:false}` - It causes the browser to work in headed mode

`chromium.launch({headless:false})` - This line returns a browser object/instance.

- `<browserobj>.newPage()` - It creates a page object

`await` - For every step await is needed

► Comment the 'projects' key-value pairs given in the configuration file to run the launching script code as required &/or expected.

Browser context - Creating multiple browser contexts / multiple users from a single browser

★ A **browser context** is like a separate, isolated browser session. It is like opening a fresh incognito window. Each context has its own cookies, local storage, cache, and session state.

- This allows one to run multiple tests in parallel without interference.
- **Multi user testing** - To check the transfer of messages between 2 or multiple users

Code Explanation:

```
import {test, chromium, webkit, firefox} from 'playwright/test';
test("Multi users", async()=>{
  const browser = await webkit.launch({headless:false});
  const windowContext1 = await browser.newContext();
  const windowContext2 = await browser.newContext();
  const page1 = await windowContext1.newPage();
  const page2 = await windowContext2.newPage();
  await page1.goto('https://www.saucedemo.com/v1/');
```

```

    await page1.fill('#user-name', 'standard_user');
    await page1.fill('#password', 'secret_sauce');
    await page1.click('#login-button');
    await page1.waitForTimeout(10000);
    await page2.goto('https://www.saucedemo.com/v1/');
    await page2.fill('#user-name', 'performance_glitch_user');
    await page2.fill('#password', 'secret_sauce');
    await page2.click('#login-button');
    await page2.waitForTimeout(10000);

    await page1.close();
    await page2.close();
  })

```

- `<browsername>.newContext()` - Creates a new browser context.
 - `<browsercontext>.newPage()` - Open a tab (page) within that context.
- `.close()` - To close the page

Selecting dropdowns, buttons

Code Explanation:

```

import {test, chromium, webkit, firefox} from 'playwright/test';
test("Launching browser", async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.facebook.com/r.php?entry_point=login');
  await page.selectOption('#month', 'Apr');
  await page.click('input[value="-1"]')
  await page.waitForTimeout(10000);
});

```

- `.selectOption('<elementlocation>', '<optionvalue>')` - To select a specific value of the dropdown list

L - 4 Switching to Elements, Multiple Browsers, Hover

25 August 2025 11:21

Switching to iframe

- `await` statement is required when an action is performed whereas to save any locator or a web element `await` statement is not required.

Code Explanation:

```
import {test, chromium, webkit, firefox} from 'playwright/test';
test("switching to iframe",async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.w3schools.com/html/html_iframe.asp');
  const frame = page.frameLocator('iframe[title="W3Schools HTML Tutorial"]');
  const sqllink = frame.locator('//a[text()='SQL"]');
  await sqllink.click();
  await page.waitForTimeout(3000);
  const javalink = page.locator('//a[text()='JAVA"]');
  await javalink.click();
  await page.waitForTimeout(3000);
});
```

- `<pagename>.frameLocator('iframelocator')` - To select iframe's elements

Switching to alert/pop up box

Alert box (pop up box) elements cannot be selected using locator function.

Code Explanation:

```
test("switching to alert",async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://testautomationpractice.blogspot.com/');
  await page.waitForTimeout(3000);

  page.on("dialog",(dialog)=>{
    console.log(dialog.message());
    dialog.accept();
    //dialog.dismiss();
  });
  await page.click("//button[text()='Simple Alert']");
});
```

- `page.on("dialog",(dialog)=>{callbackfunctioncode})` - Used for elements present on the page.
- `dialog ->` It is the object given/created by the alert box.
- `dialog.accept()` - It selects the ok button present in the pop up box.
- `dialog.dismiss()` - It selects the cancel button present in the pop up box (as in confirmation box).

Switching to new tab

Code Explanation:

```
test("switching to new tab",async()=>{
  const browser = await chromium.launch({headless:false});
  const context = await browser.newContext();
  const page = await context.newPage();
  await page.goto('https://testautomationpractice.blogspot.com/');
  //await page.waitForTimeout(3000);
  const [newTab] = await Promise.all([
    context.waitForEvent('page'),
    page.click('//button[text()='New Tab"]')
  ]);
  await newTab.screenshot({path:"photo.png"});
  await newTab.screenshot({path:"homepage.png", fullPage:true});
});
```

Procedure

- Create a web context & a page.
- Using the **concept of promise**, waiting for page loading and new tab selection is done.
- Using the concept of array destructuring, new tab object is obtained and other manipulations are done on it

- `Promise.all([arrayoffunctions&code])` - `.all` is used to execute multiple statements

- `<contextname>.waitForEvent('eventname')` - wait for the required event to happen

- `<elementname>.screenshot({path:"<path+photofilename>", <parameters>})` - Takes the screenshot of the element name & saves it at the path with the photo file name as mentioned

`fullPage:true` - Takes the screenshot of the whole web page (full scroll, without it the function takes only the page portion that visible on the screen)

Multiple Browsers

Code Explanation: All the required web browsers should be imported. The new page creation process must be carried out with all the web browsers.

```
test("launching multiple browsers",async()=>{
  const browser1 = await chromium.launch({headless:false});
  const page1 = await browser1.newPage();
  await page1.goto('https://testautomationpractice.blogspot.com/');

  const browser2 = await webkit.launch({headless:false});
  const page2 = await browser2.newPage();
  await page2.goto('https://testautomationpractice.blogspot.com/');

  const browser3 = await firefox.launch({headless:false});
  const page3 = await browser3.newPage();
  await page3.goto('https://testautomationpractice.blogspot.com/');
});
```

Hover

Code Explanation: Store the required hovering element's locator in a variable and use the function mentioned below.

- `.hover()` - Used for executing the hovering action.

```
test("hover test",async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.nike.com/in/');
  const menlink = page.locator('//a[text()='Men"]');
  await menlink.hover();
  await page.waitForTimeout(2000);
});
```

► Practice Findings

- `getByRole('menuitem', { name: 'Shoes', exact: true })`

★ `getByRole('menuitem')` - This targets an element with the **ARIA (Accessible Rich Internet Applications)** role of `menuitem`, which typically represents an item in a dropdown or navigation menu.

`{ name: 'Shoes' }` - This filters the element by its accessible name-usually derived from visible text, aria-label, or alt attributes. So it's looking for a menu item labeled "Shoes".

`{ exact: true }` - Ensures the name matches exactly. Without this, it might match partial strings like "Shoes & Accessories".

- `.locator('div:nth-child(3) > .product-card__body > figure > .product-card__link-overlay')`

★ This targets:

- The third `<div>` inside its parent (`div:nth-child(3)`)
- Inside that `<div>`, it looks for a child with class `.product-card__body`
- Then inside that, a `<figure>` element
- And finally, inside the `<figure>`, an element with class `.product-card__link-overlay`